

Trabajo Práctico Integrador Programación II

Food Store - Sistema de Gestión de Pedidos de Comida (Consola)

Alumno :

Fleitas Marcelo Hernan

Profesores Titulares

Alberto Cortez

Ariel Enferrel

Cintia Rigoni

Profesor Tutor

Jerónimo Cortez

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Programación II

25 de Junio de 2026

Tabla de contenido

Introducción y Objetivo del Trabajo	3
Desarrollo (Decisiones Técnicas y Arquitectura)	4
Imágenes del Software	6
Dificultades y Soluciones	7
Conclusiones	8
Bibliografía	9

Introducción

El presente trabajo expone el marco teórico, las decisiones de diseño y los conceptos de programación aplicados en el desarrollo del Trabajo Práctico Integrador de la asignatura Programación II. El objetivo principal consiste en describir las estructuras de datos, herramientas y principios de diseño utilizados para modelar e implementar la aplicación Food Store utilizando Java 21.

Objetivo del Trabajo

El objetivo principal del proyecto es desarrollar una aplicación capaz de gestionar los productos de un negocio gastronómico, integrando las distintas entidades que componen el sistema, tales como Categorías, Usuarios, Pedidos y Detalles de Pedido, dentro de una arquitectura de software unificada denominada Food Store.

Para ello se aplican los principios de la Programación Orientada a Objetos, permitiendo afianzar los contenidos abordados durante la asignatura tanto desde una perspectiva conceptual como práctica. Asimismo, se incorpora un sistema robusto de validaciones y manejo de excepciones que garantice la integridad de los datos y evite interrupciones inesperadas durante la ejecución del programa.

Desarrollo (Decisiones Técnicas y Arquitectura)

Durante el desarrollo del sistema se aplicaron conceptos de Programación Estructurada mediante el uso de estructuras condicionales para validar reglas de negocio y controlar el flujo de ejecución. Asimismo, se utilizaron ciclos iterativos, principalmente estructuras do-while, para mantener la persistencia de los distintos menús de consola hasta que el usuario solicite explícitamente finalizar la ejecución.

Con el objetivo de mejorar la mantenibilidad del código, se procuró respetar el principio de Responsabilidad Única (Single Responsibility Principle - SRP), garantizando que cada método y clase desempeñe una función específica dentro de la arquitectura del sistema.

Para la administración de datos en memoria se utilizaron las estructuras List y ArrayList de Java, permitiendo almacenar dinámicamente colecciones de Usuarios, Productos, Categorías, Pedidos y Detalles de Pedido.

A partir del análisis del diagrama UML provisto por la cátedra, se diseñó una arquitectura por capas compuesta por los módulos UI, Service y Entities. Esta separación permite distribuir claramente las responsabilidades de cada componente del sistema.

La capa UI concentra la interacción con el usuario mediante menús de consola y operaciones CRUD. La capa Service implementa las reglas de negocio, validaciones y coordinación entre entidades. Finalmente, la capa Entities modela las estructuras principales del dominio, encapsulando los datos y comportamientos propios de cada entidad.

Uno de los principales ejes del proyecto fue la aplicación de los conceptos de Herencia, Polimorfismo y Abstracción. Para ello se implementaron clases abstractas como Base y MenuCRUD, además de interfaces como Calculable y CrudService<T>, las cuales permiten reutilizar comportamiento común y establecer contratos de implementación para las distintas capas del sistema.

La interfaz genérica CrudService<T> define operaciones comunes para la gestión de entidades, tales como listar(), buscarPorId() y eliminar(). Gracias al uso de genéricos, la misma interfaz puede reutilizarse para diferentes tipos de objetos, como Producto, Usuario, Pedido o Categoría, evitando duplicación de código y favoreciendo el polimorfismo.

Por otra parte, la clase abstracta MenuCRUD centraliza la lógica común de navegación de los menús, permitiendo que las clases MenuProducto, MenuCategoría, MenuUsuario y MenuPedido

reutilicen la misma estructura de ejecución mediante la implementación de métodos abstractos como `listar()`, `crear()`, `editar()` y `eliminar()`.

Respecto al modelado del dominio, se respetaron las relaciones y cardinalidades definidas en el UML. Entre las principales asociaciones se destacan:

- Un Usuario puede realizar múltiples Pedidos.
- Un Pedido contiene múltiples Detalles de Pedido.
- Un Producto puede formar parte de múltiples Detalles de Pedido.
- Una Categoría agrupa múltiples Productos.

Asimismo, se implementaron relaciones bidireccionales entre Usuario-Pedido y Categoría-Producto, garantizando la consistencia de los datos mediante la sincronización de referencias entre las entidades involucradas.

Con relación a la validación de entradas y procesamiento de datos, se desarrollaron clases utilitarias especializadas. La clase `LectorConsola` centraliza las operaciones de lectura utilizando `Scanner` y captura posibles errores de entrada mediante manejo de excepciones. Complementariamente, se implementó un conjunto de utilidades de validación y normalización de texto que permiten verificar formatos, evitar datos inválidos y mejorar la consistencia de la información almacenada.

Finalmente, todas las funcionalidades desarrolladas se integran mediante un `MenuPrincipal`, desde el cual se accede a cada uno de los módulos del sistema. Posteriormente se realizaron pruebas funcionales para verificar el cumplimiento de las historias de usuario y requisitos definidos por la cátedra.

Imágenes del Software

Menú de Entrada

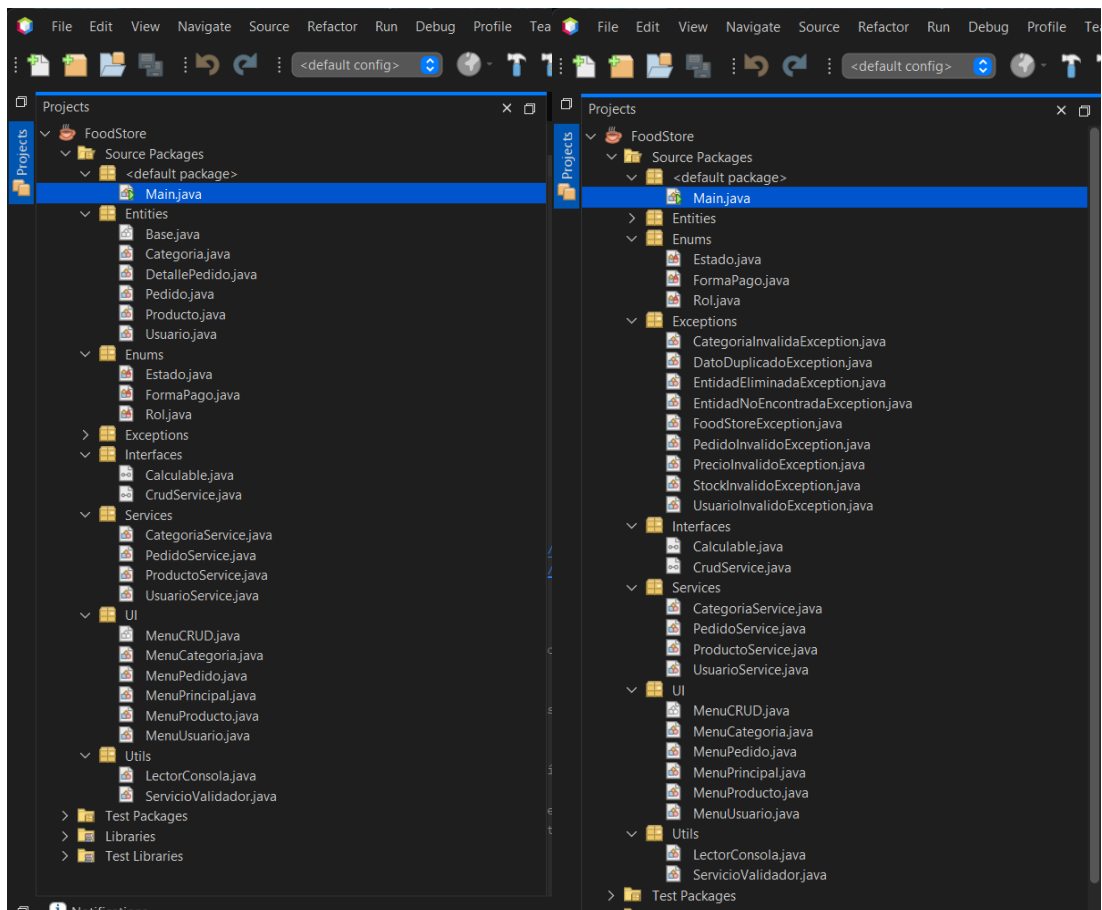
```
Output - FoodStore (run)

Datos de prueba cargados correctamente.
Categorias : 2 (IDs 1-2)
Productos : 4 (IDs 3-6)
Usuarios : 3 (IDs 7-9)
Pedidos : 3 (IDs 10-12)
  - Pedido #10: TERMINADO
  - Pedido #13: CONFIRMADO
  - Pedido #16: PENDIENTE

-----

=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorias
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione:
```

Estructura de Paquetes



Dificultades y Soluciones

Uno de los principales desafíos encontrados durante el desarrollo del proyecto fue la implementación de determinadas reglas de negocio relacionadas con la eliminación lógica de entidades.

Un caso particular surgió al gestionar la eliminación de productos que ya habían sido utilizados en pedidos registrados. Debido a que la entidad Producto no mantiene una referencia directa hacia Pedido, resultó necesario diseñar un mecanismo alternativo para validar esta restricción. La solución adoptada consistió en coordinar la información mediante las capas de servicio, permitiendo que ProductoService consulte los pedidos administrados por PedidoService antes de autorizar determinadas operaciones.

Otro aspecto relevante fue la correcta implementación de las relaciones bidireccionales definidas en el modelo UML. Para garantizar la consistencia de los datos, se implementaron métodos especializados que sincronizan ambas entidades involucradas cada vez que se crea o modifica una asociación.

Asimismo, el manejo de excepciones representó un desafío importante debido a la necesidad de mantener una ejecución estable frente a entradas inválidas o errores operativos. Para resolver esta problemática se incorporaron bloques try-catch en los puntos críticos de interacción con el usuario y se diseñó una jerarquía de excepciones personalizadas encabezada por la clase FoodStoreException.

Entre las excepciones desarrolladas se incluyen:

- CategoriaInvalidaException
- DatoDuplicadoException
- EntidadEliminadaException
- EntidadNoEncontradaException
- PedidoInvalidoException
- PrecioInvalidoException
- StockInvalidoException
- UsuarioInvalidoException

La utilización de estas excepciones permitió proporcionar mensajes de error claros y específicos, mejorando significativamente la experiencia de uso y facilitando las tareas de depuración y mantenimiento del sistema.

Conclusiones

El desarrollo del presente Trabajo Práctico Integrador permitió aplicar de forma práctica los principales conceptos abordados durante la asignatura Programación II, tales como Programación Orientada a Objetos, herencia, polimorfismo, abstracción, encapsulamiento, manejo de excepciones y utilización de colecciones dinámicas.

La implementación de una arquitectura por capas facilitó la separación de responsabilidades entre la interfaz de usuario, la lógica de negocio y las entidades del dominio, mejorando la organización y mantenibilidad del sistema.

Asimismo, la incorporación de clases abstractas, interfaces genéricas y excepciones personalizadas permitió construir una solución flexible y extensible, preparada para futuras modificaciones sin afectar significativamente la estructura general del proyecto.

Finalmente, el trabajo permitió comprender la importancia de modelar correctamente las relaciones entre entidades, validar reglas de negocio y diseñar software aplicando criterios de calidad y buenas prácticas de programación.

Bibliografía

- Oracle Corporation. (s.f.). Collection (Java Platform SE 8). Documentación oficial de Java. Recuperado de: <https://docs.oracle.com/javase/8/docs/api/java/util/Collection.html>
- Oracle Corporation. (s.f.). Scanner (Java Platform SE 8). Documentación oficial de Java. Recuperado de: <https://docs.oracle.com/javase/8/docs/api/java/util/Scanner.html>
- Oracle Corporation. (s.f.). List (Java Platform SE 8). Documentación oficial de Java. Recuperado de: <https://docs.oracle.com/javase/8/docs/api/java/util/List.html>
- Oracle Corporation. (s.f.). ArrayList (Java Platform SE 8). Documentación oficial de Java. Recuperado de: <https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html>
- Oracle Corporation. (s.f.). Generics (The Java™ Tutorials). Documentación oficial de Java. Recuperado de: <https://docs.oracle.com/javase/tutorial/java/generics/>
- Oracle Corporation. (s.f.). Exceptions. Documentación oficial de Java. Recuperado de: <https://docs.oracle.com/javase/tutorial/essential/exceptions/>
- Cimino, C. (s.f.). Interfaces GENÉRICAS en Java [Video]. YouTube. Recuperado de: <https://www.youtube.com/watch?v=HcW4W13XG5g>
- Yonkeu, S. (2024). Dissecting Layered Architecture. DEV Community. Recuperado de: <https://dev.to/yokwejuste/dissecting-layered-architecture-2ppb>